4TU Dataset/Software Monitoring Dashboard Documentation

Purpose

The monitoring dashboard is a Streamlit-based interface for inspecting recent datasets and software records available through the 4TU.ResearchData API.

It allows users to:

- Retrieve dataset or software records from 4TU.ResearchData.
- Filter records by affiliation/group, publication date range, and title keyword.
- View the results in an interactive table.
- Download the filtered results as a CSV file.
- Generate quick summary plots by group or publication date.

The dashboard is designed as a lightweight monitoring tool, not as a data editing or deposit-management tool. It only reads metadata from the 4TU.ResearchData API.

Main workflow

The monitoring workflow follows these steps:

1. Read configuration values such as the 4TU base URL, timeout, optional token, page size, and maximum number of pages.
2. Retrieve the list of 4TU groups from the `/v3/groups` API endpoint.
3. Build a lookup table that maps each group ID to its group name.
4. Retrieve articles from the `/v2/articles` API endpoint.
5. Convert the raw API response into a pandas DataFrame.
6. Let the user filter the records in the Streamlit interface.
7. Display the filtered records, summary metrics, CSV download button, diagnostics, and quick plots.

API endpoints used

`GET /v3/groups`

Used to retrieve the available groups or affiliations from 4TU.ResearchData.

The dashboard uses this endpoint to create a mapping from numeric `group_id` values to readable group names.

Example purpose:

```
group_id = 1234 -> Wageningen University and Research
```

`GET /v2/articles`

Used to retrieve datasets or software records.

The dashboard passes the following query parameters:

Parameter	Meaning
`item_type`	Selects the type of item to retrieve. The app uses `3` for datasets and `9` for software.
`published_since`	Internal lower date boundary used for the API query.
`limit`	Number of records requested per page.
`offset`	Offset used for pagination.

Item types

The dashboard currently supports two item types:

UI option	API value	Meaning
Dataset (3)	`3`	Dataset records
Software (9)	`9`	Software records

The user selects the item type in the sidebar.

Internal publication date boundary

The app uses an internal value:

```
PUBLISHED_SINCE_INTERNAL = "2000-01-01"
```

This value is passed to the API as `published_since`.

The reason for making this internal is that the user already has a clearer and more precise date filter in the Streamlit sidebar: **Publication date range**.

This gives a cleaner user experience:

- The API retrieves a broad enough set of records.
- The user filters the resulting records interactively using the date range widget.

Sidebar options

Monitoring settings

Option	Description
4TU base URL	Base URL of the 4TU.ResearchData instance. Default: `https://data.4tu.nl`.
HTTP timeout	Maximum number of seconds to wait for each API request.

Option	Description
Use Streamlit cache	Reuses previously retrieved data for the same query settings.
Refresh monitoring data	Clears the Streamlit cache and reloads data from the API.

Query settings

Option	Description
Item type	Selects whether to retrieve datasets or software.
page_size	Number of records requested per API page.
max_pages	Maximum number of API pages to retrieve.

The total maximum number of retrieved records is approximately:

```
page_size * max_pages
```

For example:

```
page_size = 100
max_pages = 3
maximum records = 300
```

Filters

Filter	Description
Affiliation/group	Filters results by the group name resolved from `group_id`.
Publication date range	Filters records by `published_date`.
Keyword in title	Performs a case-insensitive search in the title field.

Pagination logic

The function `get_recent_articles()` retrieves multiple pages from the `/v2/articles` endpoint.

For each page:

1. It calculates an offset using `page * page_size`.
2. It requests one batch of records.
3. It adds the returned records to the full result list.
4. It stops early if the API returns fewer records than requested.

This avoids unnecessary API calls when there are no more records to retrieve.

Rate-limit handling

The dashboard includes basic handling for HTTP `429 Too Many Requests` responses.

When the API returns a `429` response, the app waits and retries the request using exponential backoff:

```
Attempt 1 -> wait 1 second
Attempt 2 -> wait 2 seconds
Attempt 3 -> wait 4 seconds
Attempt 4 -> wait 8 seconds
Attempt 5 -> wait 16 seconds
```

If the API still returns rate-limit errors after the maximum number of retries, the app raises an error with this message:

```
4TU API rate limit was reached repeatedly. Try a smaller page size or a narrower published_sinc
```

In practice, the most useful setting to reduce rate-limit problems is a smaller `page_size`, for example:

```
page_size = 50 or 100
```

Caching

The app uses Streamlit caching through:

```
@st.cache_data(show_spinner=True)
```

The cached function is:

```
load_monitoring_data_cached()
```

Caching avoids repeated API calls when the same settings are used again.

The cache key depends on:

- base URL
- timeout
- item type
- internal `published_since` value
- page size
- maximum number of pages

The user can clear the cache by clicking **Refresh monitoring data**.

Data transformation

The raw API article records are converted into a pandas DataFrame with the following columns:

Column	Source	Description
`id`	API article record	Numeric 4TU item ID.

Column	Source	Description
`title`	API article record	Dataset or software title.
`published_date`	API article record	Publication date, converted to a pandas datetime value.
`group_id`	API article record	Numeric group ID.
`group_name`	Derived from `/v3/groups`	Human-readable group name.
`doi`	API article record	DOI of the item, if available.
`uuid`	API article record	UUID of the item.
`url`	API article record	Public URL of the item, if returned by the API.

The `published_date` field is converted using:

```
pd.to_datetime(..., errors="coerce")
```

Invalid or missing dates become empty date values.

Output shown in the app

The dashboard displays:

1. A metric showing the number of filtered results.
2. A table with the filtered records.
3. A download button for the filtered CSV.
4. A diagnostics panel.
5. A quick plot.

The table and CSV exclude the internal `group_id` column by default, because `group_name` is easier for users to interpret.

CSV download

The download button creates a CSV from the filtered DataFrame:

```
df.to_csv(index=False).encode("utf-8")
```

The output filename follows this pattern:

```
4tu_monitoring_item_type_<item_type>.csv
```

Examples:

```
4tu_monitoring_item_type_3.csv
4tu_monitoring_item_type_9.csv
```

Quick plots

The dashboard supports two simple plots.

Items per group

Counts the number of filtered records per affiliation/group.

Useful for answering questions such as:

- Which groups have published the most datasets?
- How many software records are associated with each group?

Items per publication date

Counts the number of filtered records by publication day.

Useful for seeing publication activity over time.

Diagnostics panel

The diagnostics expander shows:

Diagnostic	Meaning
Loaded rows	Number of rows loaded from the API before filtering.
Filtered rows	Number of rows remaining after applying sidebar filters.
Columns	List of available DataFrame columns.

This is mainly useful for debugging, testing, and workshop demonstrations.

Configuration values

The script reads several optional environment variables:

Environment variable	Default	Description
`FOURTU_BASE_URL`	`https://data.4tu.nl`	Base URL of the 4TU.ResearchData instance.
`FOURTU_TIMEOUT`	`30`	HTTP timeout in seconds.
`FOURTU_TOKEN`	empty	Optional API token. Not required for public monitoring.
`UC01_PUBLISHED_SINCE`	`2025-01-01`	Legacy/default value. The current monitoring UI uses an internal value instead.
`UC01_PAGE_SIZE`	`100`	Default number of records per API page.
`UC01_MAX_PAGES`	`3`	Default maximum number of pages to retrieve.

For Streamlit Community Cloud, avoid relying on a local `.env` file. Use Streamlit secrets only if an API token or deployment-specific setting is needed.

Recommended settings

For interactive use, recommended values are:

```
page_size = 100
max_pages = 3 to 10
```

For heavier monitoring sessions:

```
page_size = 100
max_pages = 20
```

If the API returns `429 Too Many Requests`, reduce the page size first.

Known issue in the current code

In the non-cached branch, the code currently contains this call:

```
articles = get_recent_articles(
    base_url=base_url,
    timeout=int(timeout),
    item_type=item_type,
    published_since=published_since,
    page_size=int(page_size),
    max_pages=int(max_pages),
)
```

However, `published_since` is no longer defined in the sidebar because the app now uses:

```
PUBLISHED_SINCE_INTERNAL = "2000-01-01"
```

To avoid an error when caching is disabled, replace:

```
published_since=published_since,
```

with:

```
published_since=PUBLISHED_SINCE_INTERNAL,
```

Suggested use cases

The monitoring dashboard can be used to:

- Monitor recently published WUR-related datasets.
- Inspect software records in 4TU.ResearchData.
- Export subsets of records for reporting.
- Demonstrate 4TU API usage in workshops.

- Explore publication patterns by affiliation or date.
- Prepare candidate lists for follow-up reconciliation workflows.

Limitations

The dashboard is intentionally lightweight. It does not currently:

- Edit metadata in 4TU.ResearchData.
- Deposit new datasets.
- Perform authentication-dependent curation workflows.
- Retrieve all possible metadata fields from item detail pages.
- Guarantee complete repository-wide harvesting unless page size and max pages are configured accordingly.

For large-scale or production-grade harvesting, consider moving the API access logic into a reusable Python module and adding persistent storage, logging, and scheduled execution.

Summary

The monitoring dashboard provides a simple Streamlit interface for exploring 4TU.ResearchData datasets and software records. It combines API retrieval, group-name enrichment, client-side filtering, CSV export, and quick visual summaries in a single app page.

It is especially suitable for lightweight monitoring, workshop demonstrations, and exploratory reporting.